



Instituto Superior de Engenharia de Lisboa
Departamento de Engenharia Informática
Licenciatura em Engenharia Informática e Multimédia
Sistemas Multimédia para a Internet

O SoundCloud Português

Ano Letivo:	2025/ 2026	
Docente:	Jéssica Corujeira	
Trabalho realizado por:	Guilherme Graça	N.º 51827
	Joana Ramos	N.º 50829
	Martim Ramos	N.º 51736
Data de Entrega:	12/06/2026	

Conteúdo

Índice de Ilustrações.....	1
Índice de Tabelas.....	1
Índice de Acrónimos	1
1. Introdução.....	1
1.1. Objetivos	1
1.2. Estrutura.....	1
2. Enquadramento e Trabalhos Relacionados.....	3
2.1. O SoundCloud como Referência Conceptual	3
2.2. O WordPress como Referência Técnica.....	4
2.3. Justificação das Tecnologias Escolhidas	4
3. Conceito e Tema da Plataforma.....	7
3.1. Descrição Geral	7
3.2. Tipologia de Conteúdos Multimédia	7
3.3. Organização por Categorias.....	7
3.4. Peris de Utilizador	8
4. Arquitetura do Sistema	11
4.1. Modelo Cliente/ Servidor	11
4.2. Componentes do Sistema	11
4.3. Estrutura de Ficheiros do Projeto	12
4.4. Fluxo de um Pedido HTTP	14
5. Modelo da Base de Dados	15
5.1. Visão Geral	15
5.2. Tabela ‘utilizadores’	16
5.3. Tabela ‘categorias’	16
5.4. Tabela ‘conteudo’	16
5.5. Tabela ‘estilos_musicais’	17
5.6. Tabela ‘concertos’.....	17
5.7. Tabela ‘subscrições’	17
5.8. Tabela ‘notificacoes’	18
5.9. Inicialização da Base de Dados.....	18
6. Funcionalidades Implementadas	19
6.1. Página Inicial (index.php).....	19

6.2.	Registo de Conta (registo.php)	20
6.3.	Ativação de Conta por E-mail.....	21
6.4.	Autenticação (login.php e logout.php).....	21
6.5.	Painel de Administração(admin_dashboard.php)	22
6.6.	Área do Artista e Upload de Conteúdos (artista_dashboard.php)	23
6.7.	Perfil do Artista (perfil.php)	25
6.8.	Montra de Músicas de Filtros (index.php atualizado).....	25
6.8.1.	Controlo de Visibilidade.....	26
6.8.2.	Motor de Pesquisa e Filtros	26
6.9.	Upload em Lote – ZIP + XML (artista_dashboard.php).....	26
6.9.1.	Estrutura do Pacote ZIP.....	27
6.9.2.	Estrutura do Ficheiro XML	27
6.9.3.	Processamento no Servidor	27
6.10.	Mapa de Concertos (index.php + artista_dashboard.php)	28
6.10.1.	Tecnologias Utilizadas	28
6.10.2.	Anunciar um Concerto	28
7.	Segurança	31
7.1.	Encriptação de Palavras Chave.....	31
7.2.	Princípio do Menor Privilégio	31
7.3.	Autenticação de Dois Fatores por E-mail.....	31
7.4.	Proteção Contra Spam — Google reCAPTCHA v2	31
7.5.	Isolamento de Credenciais (credenciais.php)	32
7.6.	Controlo de Acesso por Perfil.....	32
7.7.	Validação e Segurança no Upload de Ficheiros	32
8.	Discussão de Alternativas	33
8.1.	Linguagem de Servidor: PHP vs Alternativas	33
8.2.	Base de Dados: MySQL vs Alternativas.....	33
8.3.	Armazenamento de Ficheiros Multimédia: Sistema de Ficheiros vs Base de Dados	34
8.4.	Envio de E-mail: PHPMailer vs mail() Nativa.....	34
8.5.	Framework de Interface: Bootstrap vs CSS Personalizado	34
8.6.	Upload Unitário vs Upload em Lote (ZIP + XML).....	35
9.	Conclusões e Trabalho Futuro.....	37

9.1.	Conclusões.....	37
9.2.	Trabalho Futuro.....	37
10.	Bibliografia	39

Índice de Ilustrações

Figura 1 - Interface do SoundCloud original, para comparação visual com a plataforma desenvolvida	3
Figura 2 - Diagrama de perfis em escada, do acesso mínimo ao máximo.....	9
Figura 3 - Diagrama de Arquitetura Cliente/Servidor	11
Figura 4 - Estrutura de Pastas do Projeto no IDE	13
Figura 5 - Diagrama da Base de Dados	15
Figura 6 - Painel de Interesses Comum a Todos os Perfis Exceto Convidados	18
Figura 7 - Página Inicial Sem Sessão Ativa	19
Figura 8 - Página Inicial Com Sessão de Administrador Ativa	20
Figura 9 - Ecrã de Registo Com o Widget do reCAPTCHA	21
Figura 10 - Formulário de Login	22
Figura 11 - Painel de Administração.....	23
Figura 12 - Painel de Uploads de Ficheiros do Artista	24
Figura 13 - Painel de Uploads Antigos do Artista	24
Figura 14 - Painel de Perfil - Escolha do Distrito que Pode Ser Alterado	25
Figura 15 - Painel de Upload do Artista.....	27
Figura 16 - Ecrã do Formulário de Anúncio de Concerto no Dashboard do Artista	29
Figura 17 - Ecrã Com o Mapa Leaflet.js e os Pins de Concerto Visíveis	29

Índice de Tabelas

Tabela 1 - Comparação Entre as Funcionalidades do WordPress e do Projeto	4
Tabela 2 -Principais Ficheiros e Pastas do Projeto e as Suas Funções	12
Tabela 3 - Proteções Implementadas e o Seu Objetivo	32

Índice de Acrónimos

BD	Base de Dados
CSS	<i>Cascading Style Sheet</i>
EP	<i>Extended Play</i>
HTML	<i>Hypertext Markup Language</i>
HTTP	<i>Hypertext Transfer Protocol</i>
JPG	<i>Joint Photographic Exp+erts Group</i>
MP3	<i>Moving Picture Experts Group Audio Layer 3</i>
PHP	<i>Hypertext Preprocessor</i>
PNG	<i>Portable Network Graphics</i>
SGC	Sistema de Gestão de Conteúdo
SMTP	<i>Simple Mail Transfer Protocol</i>
SQL	<i>Structured Query Language</i>
WAV	<i>Waveform Audio File Format</i>

1. Introdução

O presente relatório foi desenvolvido no âmbito da unidade curricular de Sistemas de Multimédia para a Internet, e descreve o processo de conceção, planeamento e implementação de um SGC multimédia com base numa arquitetura cliente/servidor.

O sistema desenvolvido tem como tema uma plataforma de partilha multimédia dedicada a artistas musicais independentes portugueses, inspirada no conceito do **SOUNDCLOUD**, mas com foco exclusivo na música local e regional. A plataforma permite que músicos publiquem as suas obras (ficheiros de áudio e imagens de capa) organizadas por distrito de origem, e que os fãs descubram e explorem esses conteúdos de forma filtrada.

1.1. Objetivos

Os principais objetivos do trabalho são:

- Desenvolver um SGC funcional em PHP com suporte a múltiplos perfis de utilizador com diferentes níveis de permissão;
- Implementar um sistema de autenticação seguro com verificação de conta por e-mail (autenticação de dois fatores);
- Permitir o upload e gestão de conteúdos multimédia (áudio e imagem) com controlo de visibilidade público/privado;
- Organizar os conteúdos através de um sistema de categorias hierárquico (categorias principais geridas por administradores e categorias secundárias definidas pelos artistas);
- Garantir a segurança do sistema através de boas práticas de desenvolvimento *web*.

1.2. Estrutura

O presente documento está organizado da seguinte forma: o **CAPÍTULO 2 - ENQUADRAMENTO E TRABALHOS RELACIONADOS** apresenta o enquadramento do projeto e a comparação com sistemas similares existentes; o **CAPÍTULO 3 - CONCEITO E TEMA DA PLATAFORMA** descreve o conceito e tema da plataforma; o **CAPÍTULO 4 - ARQUITETURA DO**

SISTEMA detalha a arquitetura técnica do sistema; o **CAPÍTULO 5 - MODELO DA BASE DE DADOS** apresenta o modelo da base de dados; o **CAPÍTULO 6 - FUNCIONALIDADES IMPLEMENTADAS** descreve as funcionalidades implementadas; o **CAPÍTULO 7 - SEGURANÇA** aborda as medidas de segurança adotadas; o **CAPÍTULO 8 - DISCUSSÃO DE ALTERNATIVAS** discute alternativas às opções tomadas; e o **CAPÍTULO 9 - CONCLUSÕES E TRABALHO FUTURO** apresenta as conclusões e trabalho futuro.

2. Enquadramento e Trabalhos Relacionados

O desenvolvimento de plataformas de partilha de conteúdos multimédia na web não é um conceito novo. Existem hoje diversas soluções consolidadas no mercado que servem de referência para este projeto, tanto a nível conceptual como técnico.

2.1. O SoundCloud como Referência Conceptual

O SoundCloud é uma plataforma online de distribuição de áudio que permite a qualquer utilizador fazer *upload*, partilhar e descobrir música. Lançado em 2008, tornou-se uma das principais ferramentas de promoção para artistas independentes a nível mundial. A plataforma suporta múltiplos perfis de utilizador – criadores de conteúdo e ouvintes- e organiza os conteúdos através de etiquetas (*tags*) e géneros musicais.

A plataforma desenvolvida neste projeto partilha o mesmo propósito base, dar voz a artistas independentes, mas distingue-se por dois aspetos fundamentais: o foco exclusivo no panorama musical português e a organização geográfica por distrito, que aproxima os artistas do seu público local.

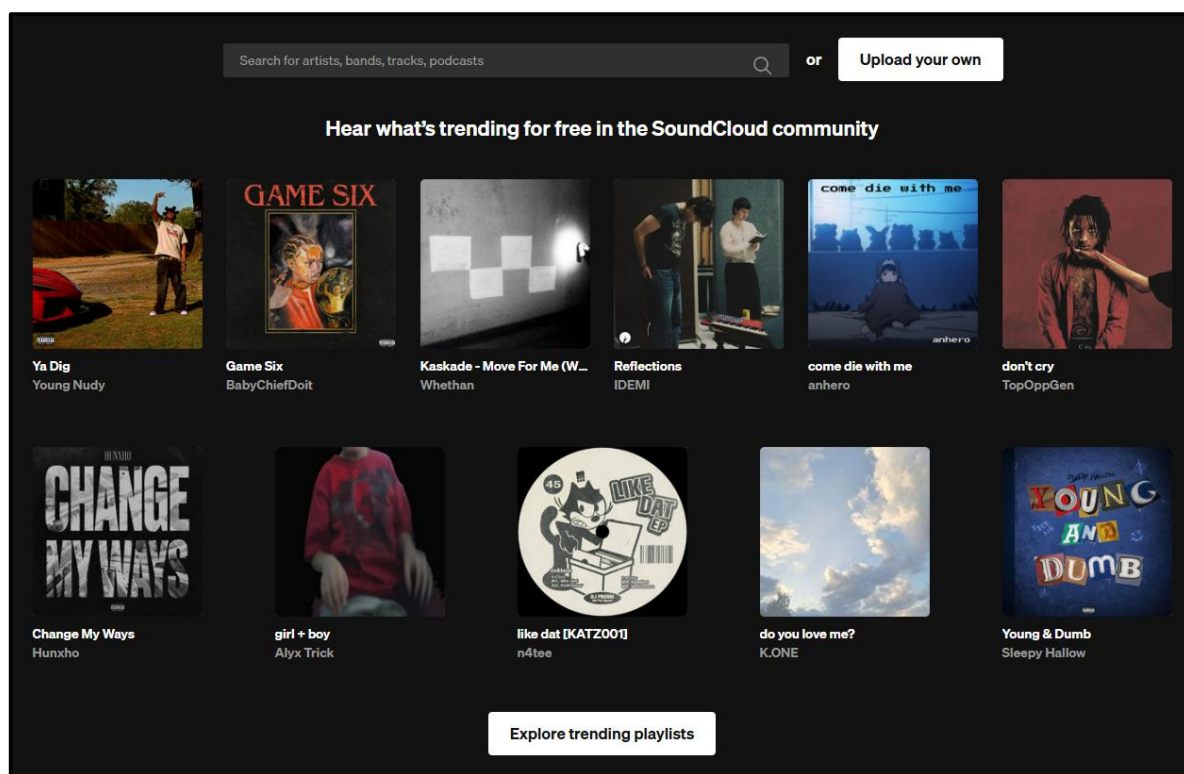


Figura 1 - Interface do SoundCloud original, para comparação visual com a plataforma desenvolvida

2.2. O WordPress como Referência Técnica

Do ponto de vista técnico, o **WordPress** é o SGC mais utilizado no mundo e serve como referência direta para a arquitetura desse projeto. O WordPress assenta exatamente na mesma estrutura exigida pelo enunciado: utiliza **PHP** como linguagem de servidor, **Apache** como servidor *web* e o sistema de ficheiros do servidor para armazenar os conteúdos multimédia.

A **ERRO! A ORIGEM DA REFERÊNCIA NÃO FOI ENCONTRADA.** apresenta comparação direta entre as funcionalidades do WordPress e as funcionalidades implementadas neste projeto.

Tabela 1 - Comparação Entre as Funcionalidades do WordPress e do Projeto

	WordPress	SoundCloud PT
Linguagem do Servidor	PHP	PHP
Servidor Web	Apache	Apache
Base de Dados	MySQL	MySQL
Armazenamento de Ficheiros	Sistema de ficheiros locais	Sistema de ficheiros local (uploads/)
Perfis de Utilizador	Visitante, Subscritor, Autor, Administrador	Convidado, Utilizador, Simpatizante, Administrador
Upload de Multimédia	Biblioteca Multimédia (imagens, vídeo, áudio)	Áudio (MP3/WAV) e Imagem (JPG/PNG)
Categorias	Categorias (fixas) + <i>Tags</i> (livres)	Categorias principais / Distritos (Admin) + Categorias secundárias/ Estilos (Artista)
Controlo de Visibilidade	Público, Privado, Protegido por senha	Público, Privado
Autenticação	Username + Password	Email + Password + Verificação 2FA por e-mail
Anti-Spam	Plugín externo	Google reCAPTCHA v2 integrado
Foco Temático	Genérico (blogs, sites, lojas)	Música independente portuguesa por distrito

A principal diferença reside no facto de o WordPress ser uma solução genérica de gestão de conteúdos textuais, enquanto o sistema desenvolvido foi contruído de raiz para responder a um caso de uso específico: a partilha de conteúdos multimédia por perfis com papéis bem definidos e com uma organização geográfica integrada.

2.3. Justificação das Tecnologias Escolhidas

A escolha das tecnologias seguiu os requisitos definidos no enunciado, mas foi também sustentada por critérios técnicos objetivos. O PHP é uma linguagem amplamente suportada em servidores web partilhados, com vasta documentação e integração nativa com **MySQL**. O MySQL é um sistema de gestão de base de dados relacional robusto, gratuito e ideal para estruturas de dados como as que este projeto exige: utilizadores, conteúdos e categorias com relações entre si. O **Bootstrap** foi adotado como *framework* de interface para garantir um *design* responsivo sem necessidade de desenvolver CSS de raiz, tornando a plataforma acessível tanto em *desktop* como em dispositivos móveis.

3. Conceito e Tema da Plataforma

3.1. Descrição Geral

A plataforma desenvolvida, denominada de **SoundCloud Português**, é um sistema de gestão e partilha de conteúdos multimédia dedicado exclusivamente a artistas musicais independentes portugueses. O conceito surgiu de necessidade de criar um espaço digital onde músicos de garagem, produtos e bandas emergentes possam publicar e promover o seu trabalho, sem competir com artistas *mainstream* de grandes editoras.

O foco geográfico é um dos elementos diferenciadores da plataforma. Toda a organização de conteúdos assenta numa lógica regional, os distritos de Portugal, o que permite que um fã de música do Porto encontre facilmente artistas e eventos da sua zona, e que um artista de Lisboa saiba exatamente onde o seu trabalho está a ser publicado e para que audiência.

3.2. Tipologia de Conteúdos Multimédia

A plataforma suporta dois tipos de conteúdos multimédia:

- **Áudio:** Ficheiros MP3 ou WAV correspondentes a músicas originais, *beats* ou gravações acústicas;
- **Imagem:** Ficheiros JPG e PNG utilizados como capas de *singles*, cartazes de concertos ou fotografias de bastidores.

Esta escolha cobre os dois tipos de multimédia mais relevantes para o contexto musical e cumpre o requisito mínimo de suporte a pelo menos dois tipos de conteúdo multimédia distintos.

3.3. Organização por Categorias

O sistema de categorias é hierárquico e divide-se em dois níveis:

- **Categorias Principais – Distritos:** definidas exclusivamente pelos Administradores da plataforma. Representam os distritos de Portugal (Lisboa, Porto, Faro, Coimbra, Braga, entre outros) e garantem que toda a música publicada fica associada a uma região geográfica concreta. A base de dados é inicializada com cinco distritos de exemplo, podendo o administrador adicionar mais através do painel de controlo.

- **Categorias Secundárias – Estilos Musicais:** definidas livremente pelos Simpatizantes (artistas) no momento do *upload*. Exemplos incluem Trap, Punk Rock, Fado Acústico ou Hip-Hop. Esta liberdade criativa permite que os próprios artistas classifiquem o seu trabalho de forma orgânica.

3.4. Peris de Utilizador

A plataforma define quatro perfis de utilizador com permissões progressivas, mapeados para os diferentes intervenientes do ecossistema musical:

- **Convidado:** qualquer pessoa que aceda à plataforma sem autenticação. Pode visualizar e ouvir conteúdos públicos e realizar pesquisas gerais. Equivale ao ouvinte casual que chegou à plataforma através de um *link* partilhado nas redes sociais.
- **Utilizador (Fã):** utilizador registado e com conta verificada. Para além de visualizar conteúdos, pode gerir as suas preferências e subscrever notificações por distrito ou estilo musical. Equivale ao fã fiel ou ao olheiro de uma rádio local.
- **Simpatizante (Artista):** utilizador com permissões de criação de conteúdo. Pode fazer upload de ficheiros de áudio e imagem, associar os seus conteúdos a um distrito e a um estilo musical, e definir a visibilidade de cada publicação (público ou privado). Equivale ao músico independente, produtor ou manager de banda.
- **Administrador:** nível máximo de acesso. Gere os utilizadores da plataforma, promove perfis (por exemplo, transforma um Utilizador em Simpatizante), e define as categorias principais. Equivale à equipa responsável pela operação da plataforma.

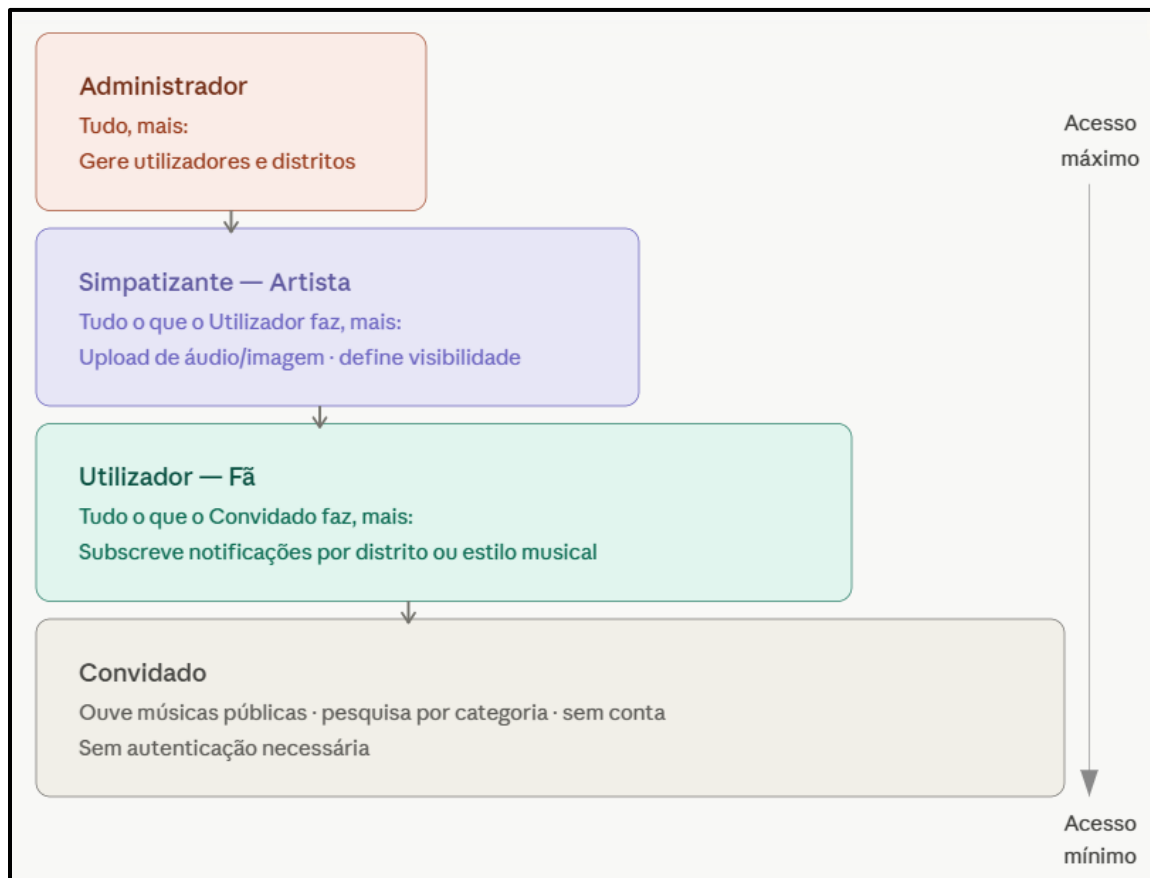


Figura 2 - Diagrama de perfis em escada, do acesso mínimo ao máximo

4. Arquitetura do Sistema

4.1. Modelo Cliente/ Servidor

O sistema assenta numa arquitetura cliente/servidor clássica, onde o cliente (o browser do utilizador) comunica com o servidor através do protocolo HTTP. O servidor é responsável por processar a lógica da aplicação, comunicar com a base de dados e devolver as páginas HTML geradas dinamicamente ao cliente.

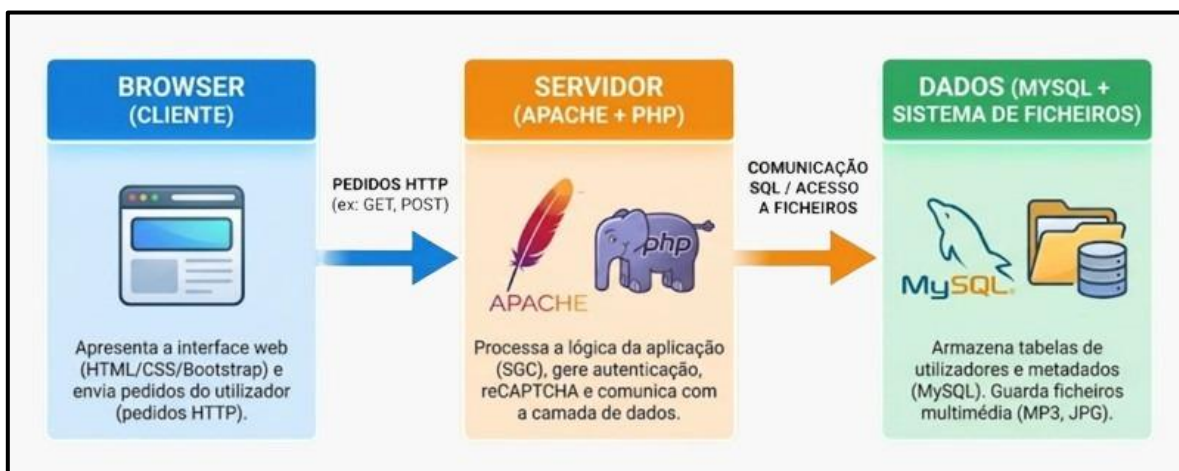


Figura 3 - Diagrama de Arquitetura Cliente/Servidor

4.2. Componentes do Sistema

O sistema é composto por quatro camadas principais:

- **Servidor Web (Apache):** responsável por receber os pedidos HTTP do browser e encaminhá-los para os ficheiros PHP correspondentes. Foi utilizado em conjunto com Docker para garantir um ambiente de desenvolvimento isolado e reproduzível.
- **Linguagem de Servidor (PHP):** responsável por toda a lógica da aplicação. Processa formulários, valida dados, comunica com a base de dados, gere sessões de utilizador e gera as páginas HTML que são enviadas ao *browser*. A ligação à base de dados é gerida pelo ficheiro `conexao.php`, que centraliza as credenciais de acesso ao MySQL.
- **Base de Dados (MySQL):** armazena toda a informação textual e relacional do sistema: utilizadores, conteúdos multimédia e categorias. A base de dados foi

criada com `charset utf8mb4` para suporte nativo a caracteres portugueses com acentuação.

- **Sistema de Ficheiros (Pasta `uploads/`)** : os ficheiros multimédia (MP3, WAV, JPG, PNG) não são guardados na base de dados. Em vez disso, são guardados fisicamente na pasta `uploads/` do servidor, e a base de dados regista apenas o caminho (*path*) para esse ficheiro. Esta é a mesma abordagem utilizada pelo *WordPress* e pela generalidade dos SGC modernos.

4.3. Estrutura de Ficheiros do Projeto

Tabela 2 - Principais Ficheiros e Pastas do Projeto e as Suas Funções

Ficheiro / Pasta	Função
<code>index.php</code>	Página inicial, deteta sessão e redireciona
<code>login.php</code>	Autenticação de utilizadores
<code>registo.php</code>	Criação de novas contas com reCAPTCHA e 2FA
<code>logout.php</code>	Destruição segura da sessão
<code>ativar_conta.php</code>	Validação do <i>token</i> de ativação por e-mail
<code>admin_dashboard.php</code>	Painel restrito a administradores
<code>artista_dashboard.php</code>	Painel de <i>upload</i> restrito a simpatizantes
<code>conexao.php</code>	Ponte de ligação ao MySQL
<code>credenciais.php</code>	Isolamento de chaves privadas e APIs
<code>Style/style.css</code>	Folha de estilos global
<code>PHPMailer/</code>	Biblioteca para envio de e-mails via SMTP
<code>database/base_dados.sql</code>	<i>Script</i> de criação e inicialização da base de dados
<code>interesses.php</code>	Gestão das subscrições e interesses dos utilizadores para receber notificações de concertos
<code>perfil.php</code>	Gestão e atualização do perfil do utilizador
<code>ler_notificacoes.php</code>	Marca notificações como lidas e atualiza o estado das notificações do utilizador
<code>database/migracao_capa.sql</code>	Migração da base de dados para suporte de capas/imagens dos conteúdos
<code>database/migracao_concertos.sql</code>	Migração para criação/ alteração das tabelas relacionadas com os concertos
<code>database/migracao_descricao.sql</code>	Migração para adicionar descrições aos registos da aplicação
<code>database/migracao_estilos.sql</code>	Migração para criação/ atualização da estrutura de estilos musicais
<code>database/migracao_susbscricao.sql</code>	Migração para suporte ao sistema de subscrições/ interesses
<code>Style/img/</code>	Recursos gráficos utilizados pela interface da aplicação

uploads/temp_*	Diretórios temporários usados durante o processo de upload e processamento de ficheiros
exemplo_metadados.sql	Exemplo de ficheiro XML que contém meta dados de conteúdos multimédia
uploads/	Armazenamento permanente e temporário de ficheiros multimédia enviados pelos artistas.

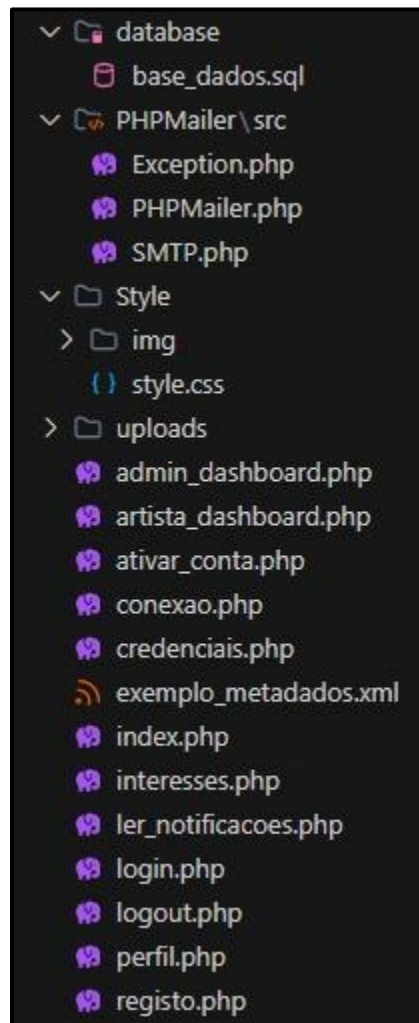


Figura 4 - Estrutura de Pastas do Projeto no IDE

4.4. Fluxo de um Pedido HTTP

Quando um utilizador acede a uma página da plataforma, o fluxo é o seguinte: o browser envia um pedido HTTP ao Apache; o Apache identifica o ficheiro PHP correspondente e passa-o ao interpretador PHP; o PHP executa a lógica, consulta o MySQL se necessário e lê ou escreve ficheiros na pasta `uploads/`; por fim, o PHP gera o HTML final que é devolvido ao browser e apresentado ao utilizador.

5. Modelo da Base de Dados

5.1. Visão Geral

A base de dados da plataforma, denominada `bd_soundcloud_pt`, foi criada com o `charset utf8mb4` para garantir suporte nativo a caracteres portugueses com acentuação. É composta por três tabelas principais: utilizadores, categorias e conteúdos, que estabelecem entre si relações de chave estrangeira para garantir a integridade referencial dos dados.

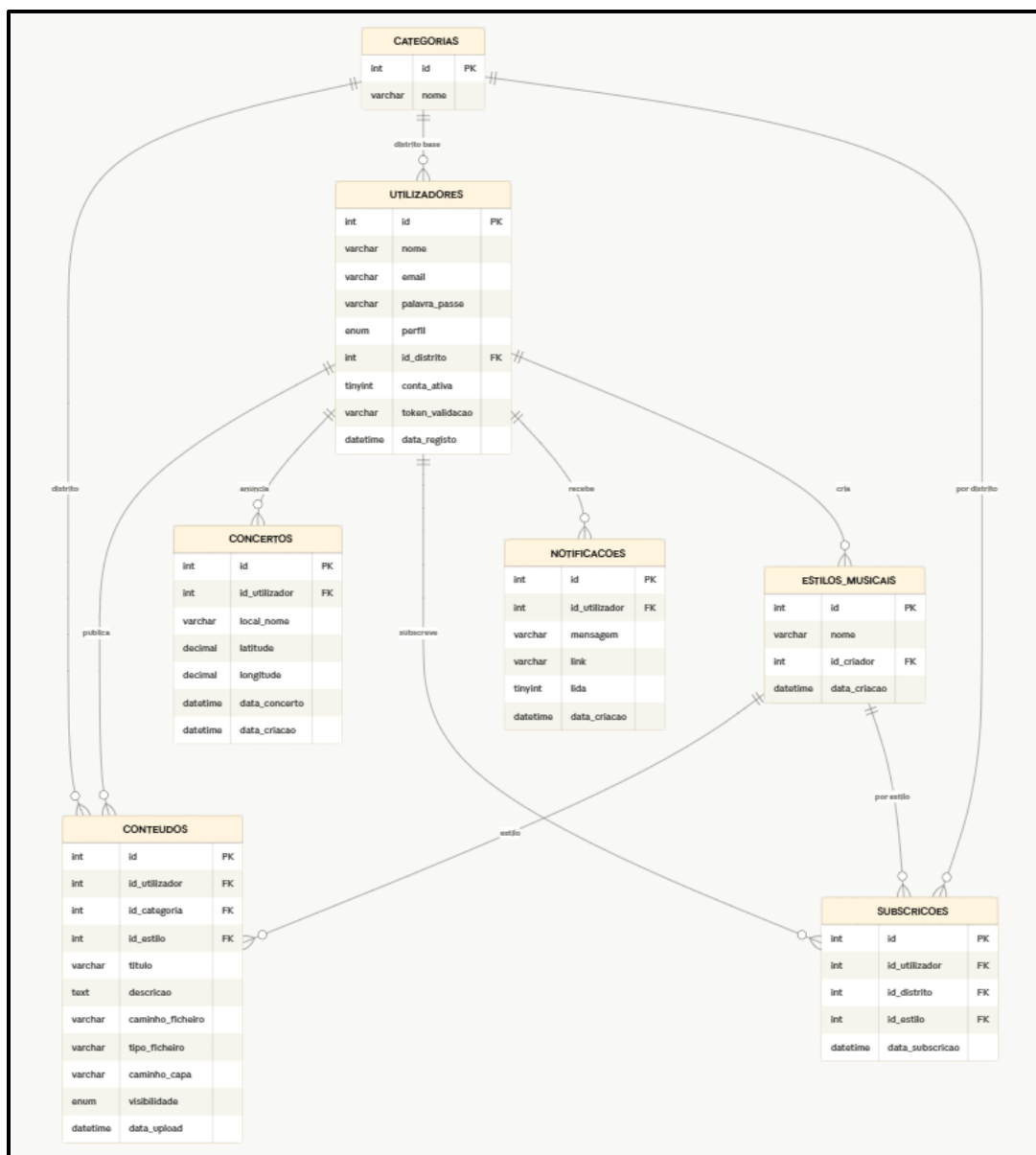


Figura 5 - Diagrama da Base de Dados

5.2. Tabela ‘utilizadores’

Armazena todas as contas registadas na plataforma. Os campos mais relevantes são o perfil, definido como um tipo `ENUM` com os valores administrador, simpatizante e utilizador, que determina as permissões de cada conta; o campo `conta_ativa`, que impede o *login* enquanto a conta não estiver verificada por e-mail; e o `token_validacao`, um código único gerado no momento do registo e enviado por e-mail, que é apagado da base de dados após a ativação da conta.

Na fase atual do projeto, foi adicionado o campo `id_distrito`, uma chave estrangeira que referencia a tabela `categorias` e que representa o Distrito Base do artista. Este campo é preenchido obrigatoriamente através da página `perfil.php` antes do Simpatizante poder efetuar o seu primeiro *upload*, sendo `NULL` para utilizadores que não sejam artistas.

5.3. Tabela ‘categorias’

Armazena as categorias principais da plataforma, que correspondem aos distritos de Portugal. O campo nome tem a restrição `UNIQUE`, impedindo a inserção de categorias duplicadas. A tabela é inicializada com cinco distritos: Porto, Lisboa, Faro, Coimbra e Braga, podendo ser expandida pelo administrador através do painel de controlo.

5.4. Tabela ‘conteudo’

Armazena os registos de cada ficheiro multimédia publicado na plataforma. Relaciona-se com utilizadores através de `id_utilizador` e com categorias através de `id_categoria`, ambas chaves estrangeiras com as seguintes políticas de integridade: se um utilizador for eliminado, os seus conteúdos são eliminados em cascata (`ON DELETE CASCADE`); se uma categoria for eliminada, a operação é bloqueada enquanto existirem conteúdos associados (`ON DELETE RESTRICT`). O campo `caminho_ficheiro` armazena o caminho relativo para o ficheiro físico guardado na pasta `uploads/` do servidor, e o campo visibilidade é um `ENUM` com os valores `publico` e `privado`.

Foram ainda adicionados os campos `id_estilo`, chave estrangeira para a tabela `estilos_musicais` que classifica o género musical da faixa, e `caminho_capa`, que armazena o caminho relativo para o ficheiro de imagem de capa associado ao conteúdo de áudio.

5.5. Tabela ‘estilos_musicais’

Armazena as categorias secundárias da plataforma, correspondentes aos géneros musicais definidos pelos próprios artistas. O campo `nome` tem a restrição `UNIQUE`, impedido duplicados. Ao contrário das categorias principais, os estilos musicais podem ser criados em tempo real pelos Simpatizantes no momento do *upload*, sem necessidade de intervenção do administrador.

5.6. Tabela ‘concertos’

Armazena os eventos musicais anunciados pelos artistas na plataforma. Para além do nome do local em texto livre e da data e hora do evento, guarda as coordenadas geográficas (`latitude` e `longitude`) resolvidas automaticamente pelo serviço **NOMINATIM**¹ no momento do anúncio. Relaciona-se com utilizadores através de `id_utilizador` com a política `ON DELETE CASCADE`, eliminando automaticamente os concertos de um artista caso a sua conta seja removida.

5.7. Tabela ‘subscrições’

Armazena as preferências de notificação dos utilizadores registados. Cada registo representa uma subscrição de um utilizador a um distrito, a um estilo musical, ou a ambos em simultâneo. Os campos `id_distrito` e `id_estilo` são ambos opcionais (`DEFAULT NULL`), permitindo que um utilizador subscreva apenas um distrito sem estilo associado, apenas um estilo sem distrito, ou a combinação de ambos. Ambas as chaves estrangeiras têm a política `ON DELETE CASCADE`, garantindo que as subscrições são automaticamente removidas caso o distrito ou estilo musical a que se referem seja eliminado.

¹ O nominatim é uma ferramenta de pesquisa geográfica *open source* que pesquisa dados do OpenStreetMap. Ela também converte texto. Por exemplo "Coliseu dos Recreios, Lisboa" em latitude/longitude dessa mesma localização.

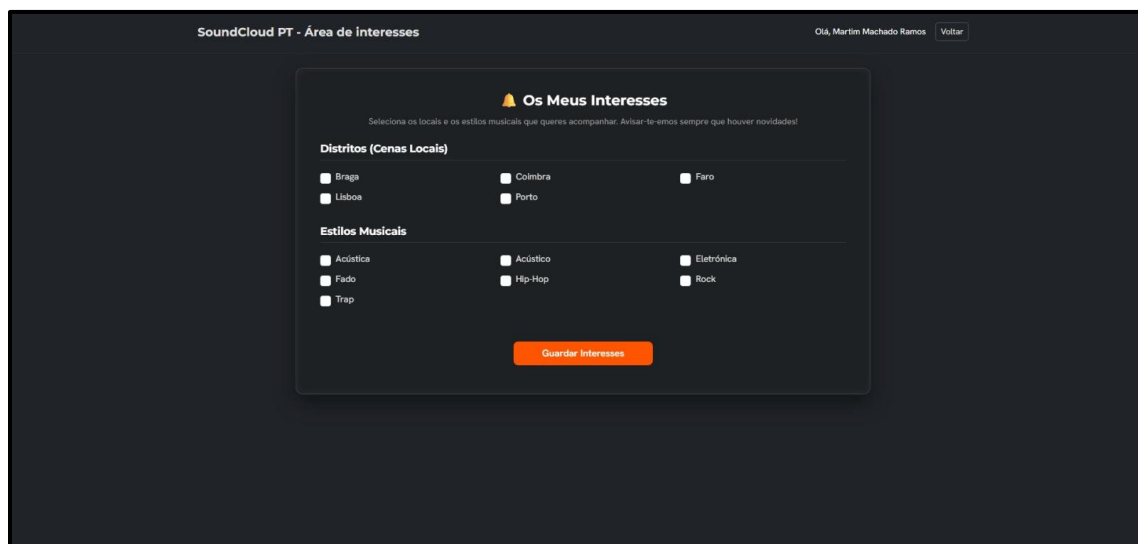


Figura 6 - Painel de Interesses Comum a Todos os Perfis Exceto Convidados

5.8. Tabela ‘notificacoes’

Armazena as notificações *in-app* geradas pelo sistema para cada utilizador. Quando um artista publica um novo conteúdo num distrito ou estilo musical que um utilizador subscreveu, o sistema gera automaticamente um registo nesta tabela com uma mensagem descritiva e um *link* opcional para o conteúdo em questão. O campo *lida*, do tipo `TINYINT`, funciona como um booleano que indica se o utilizador já visualizou a notificação (1) ou não (0). Tal como as restantes tabelas dependentes de utilizadores, aplica a política `ON DELETE CASCADE`, eliminando todas as notificações de uma conta caso esta seja removida.

5.9. Inicialização da Base de Dados

O ficheiro `base_dados.sql` contém o script completo de criação e inicialização da base de dados, incluindo a criação das sete tabelas (`utilizadores`, `categorias`, `estilos_musicais`, `conteudos`, `concertos`, `subscricoes` e `notificacoes`), a inserção dos cinco distritos iniciais nas categorias, e a inserção de um utilizador administrador pré-configurado para permitir o primeiro acesso à plataforma. Note-se que, para remover o utilizador administrador inicial, é necessário primeiro dissociar os registos que lhe estão associados nas restantes tabelas, nomeadamente os distritos e estilos musicais inseridos por omissão."

6. Funcionalidades Implementadas

6.1. Página Inicial (index.php)

A página inicial funciona como ponto de entrada da plataforma e como *hub* de navegação. Ao carregar, deteta automaticamente se existe uma sessão ativa através da variável `$_SESSION`. Se o utilizador não estiver autenticado, apresenta dois botões, "Iniciar Sessão" e "Criar Nova Conta". Se a sessão estiver ativa, saúda o utilizador pelo nome, exibe o seu perfil atual e apresenta o botão de acesso ao painel correspondente ao seu nível de permissão, painel de administração para administradores, área de artista para simpatizantes.

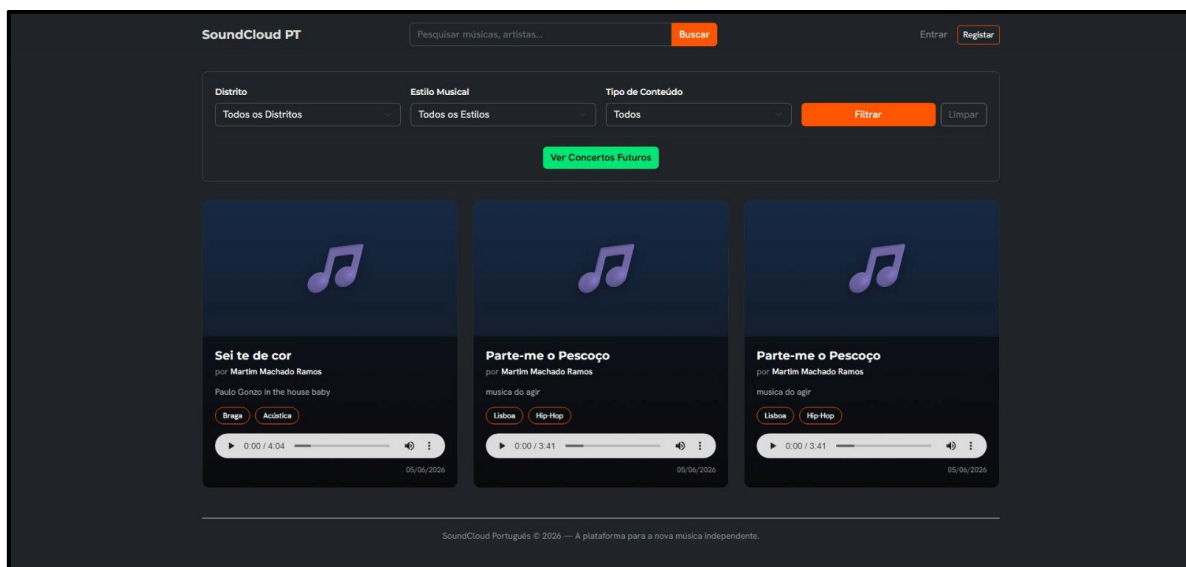


Figura 7 - Página Inicial Sem Sessão Ativa

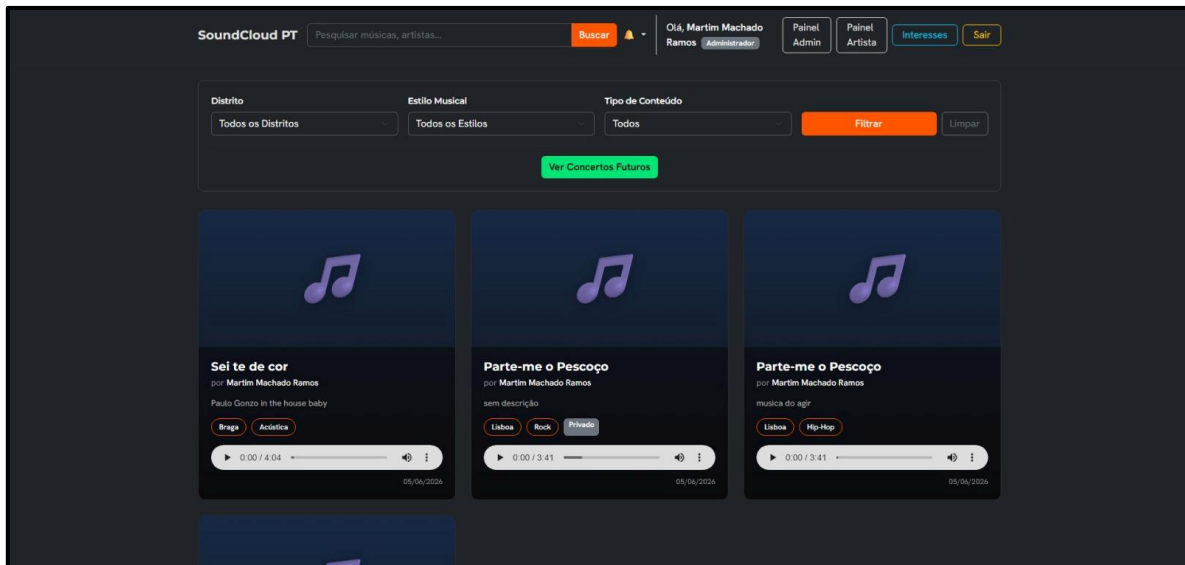


Figura 8 - Página Inicial Com Sessão de Administrador Ativa

6.2. Registo de Conta (registo.php)

O formulário de registo solicita ao utilizador o nome ou nome da banda, o endereço de e-mail e a palavra-passe. O processo de criação de conta envolve as seguintes etapas:

1. Validação do lado do servidor de todos os campos obrigatórios;
2. Verificação do *token* *reCAPTCHA* junto à API da Google, para confirmar que o pedido provém de um humano;
3. Encriptação da palavra-passe com o algoritmo *bcrypt* através da função `password_hash()`;
4. Inserção do registo na base de dados com o perfil mínimo utilizador e a conta marcada como inativa (`conta_ativa = 0`);
5. Geração de um *token* único de 64 caracteres com `bin2hex(random_bytes(32))`;
6. Envio automático de um e-mail de ativação com o link de confirmação, através da biblioteca PHPMailer via SMTP.

Enquanto a conta não for ativada pelo e-mail, o *login* é bloqueado, garantindo que todos os endereços de e-mail registados são válidos e pertencem ao utilizador.

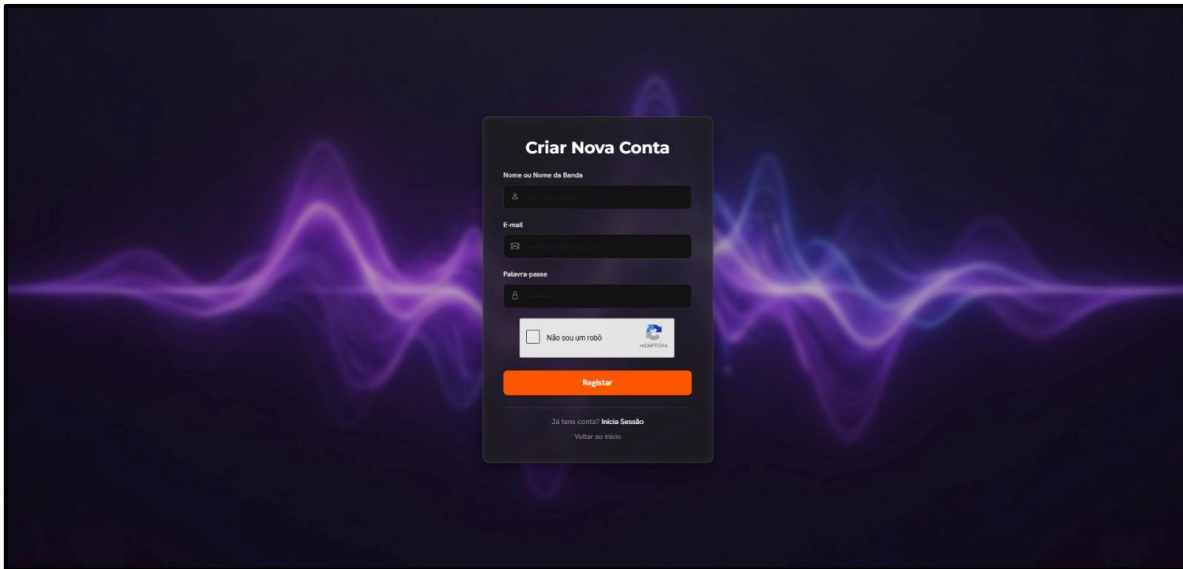


Figura 9 - Ecrã de Registo Com o Widget do reCAPTCHA

6.3. Ativação de Conta por E-mail

Ao clicar no *link* recebido no e-mail, o utilizador é redirecionado para `ativar_conta.php`, que recebe o *token* como parâmetro `GET`. O sistema pesquisa na base de dados um utilizador inativo com esse *token* exato. Se encontrado, atualiza o campo `conta_ativa` para 1 e limpa o campo `token_validacao`, invalidando o *link* de ativação para utilizações futuras. Se o *token* não for encontrado ou já tiver sido usado, é apresentada uma mensagem de erro adequada.

6.4. Autenticação (`login.php` e `logout.php`)

O formulário de login recebe o e-mail e a palavra-passe do utilizador. O processo de autenticação segue os seguintes passos:

1. Consulta à base de dados pelo e-mail introduzido;
2. Verificação da palavra-passe com `password_verify()`, que compara com o *hash* guardado;
3. Verificação de que a conta está ativa (`conta_ativa = 1`);
4. Em caso de sucesso, armazenamento do `id`, `nome` e `perfil` do utilizador na sessão PHP, e redirecionamento para a página inicial.

O *logout* é tratado pelo ficheiro `logout.php`, que destrói todas as variáveis de sessão com `session_destroy()`, garantindo uma saída completamente segura.

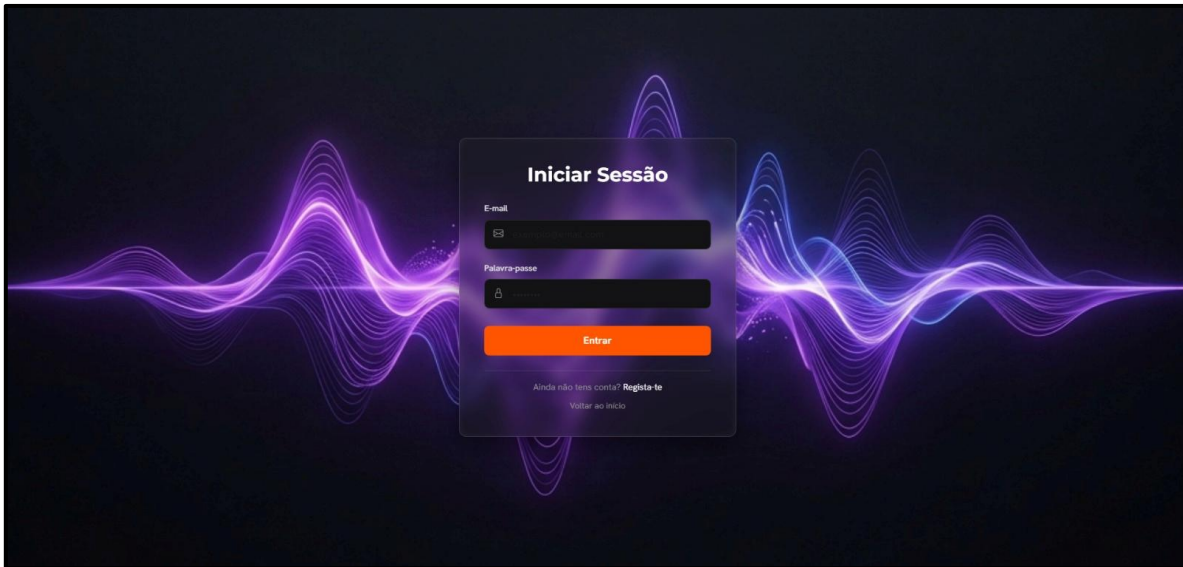


Figura 10 - Formulário de Login

6.5. Painel de Administração(admin_dashboard.php)

O painel de administração é de acesso restrito, qualquer tentativa de acesso direto por um utilizador sem o perfil `administrador` é imediatamente redirecionada para a página inicial. O painel é composto por duas secções:

Gestão de Utilizadores: apresenta uma tabela com todos os utilizadores registados, incluindo o seu estado de conta (ativa ou inativa) e o perfil atual. O administrador pode alterar o perfil de qualquer utilizador através de um menu *dropdown* com as opções `utilizador`, `simpatizante` e `administrador`. Existe uma salvaguarda que impede o administrador de alterar o seu próprio perfil, evitando que perca acidentalmente o acesso ao painel.

Gestão de Categorias: apresenta a lista de todos os distritos existentes e um formulário para adicionar novos. O campo `nome` é validado como `UNIQUE` na base de dados, impedindo a criação de categorias duplicadas.

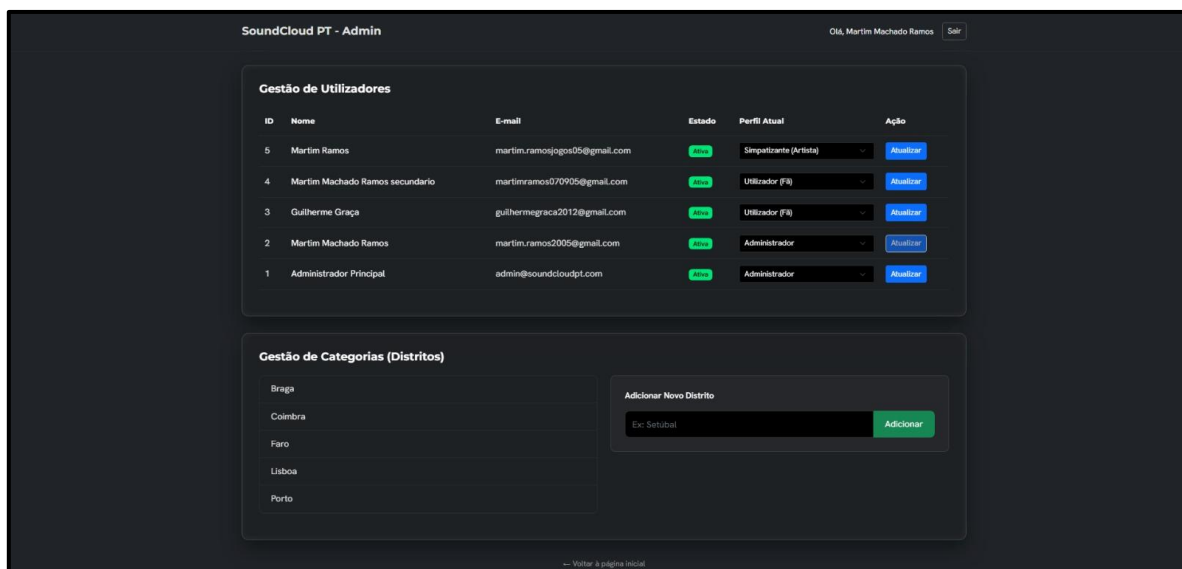


Figura 11 - Painel de Administração

6.6. Área do Artista e Upload de Conteúdos (artista_dashboard.php)

A área do artista é acessível a utilizadores com perfil `simpatizante` ou `administrador`. Disponibiliza um formulário de upload unitário que permite ao artista publicar um ficheiro de cada vez, associando-lhe os seguintes metadados:

- **Título** da música ou da capa;
- **Categoria (Distrito):** selecionada a partir das categorias principais definidas pelos administradores;
- **Visibilidade:** `Publico` (acessível a qualquer visitante) ou `Privado` (restrito a utilizadores com sessão ativa);
- **Ficheiro:** aceita os formatos MP3 e WAV para áudio, e JPG e PNG para imagem.

SoundCloud PT - Área do Artista

O Teu Distrito Base: **Lisboa** — Todas as tuas músicas são publicadas neste distrito. [Alterar](#)

Novo Lançamento (Upload Unitário)

Título da Música ou Capa
Ex: Batida de Fado Acústico

Estilo Musical
— Sem estilo específico —
Não encontras o teu estilo? [Cria um novo](#) →

Descrição
Fala-nos um pouco sobre este lançamento...

Privacidade / Visibilidade
Público (Visível para toda a gente)

Ficheiro Multimédia (Áudio/Imagem)
Escolher ficheiro Nenhum ficheiro selecionado

Imagem de Capa (Opcional)
Escolher ficheiro Nenhum ficheiro selecionado
Associar uma capa específica a esta música.

[Enviar Ficheiro](#)

Lançamento em Lote (ZIP + XML)

Envia várias músicas e capas de uma só vez! Prepara um ficheiro `.zip` contendo os ficheiros multimédia e um ficheiro `entidades.xml` com a informação de cada faixa.

[Ver exemplo de ficheiro XML](#)

Ficheiro ZIP
Escolher ficheiro Nenhum ficheiro selecionado

O ZIP deve conter os ficheiros multimédia (MP3/WAV/JPG/PNG) e um ficheiro `entidades.xml`.

[Enviar Lote \(ZIP\)](#)

Figura 12 - Paine de Uploads de Ficheiros do Artista

Envia várias músicas e capas de uma só vez! Prepara um ficheiro `.zip` contendo os ficheiros multimédia e um ficheiro `entidades.xml` com a informação de cada faixa.

[Ver exemplo de ficheiro XML](#)

Ficheiro ZIP
Escolher ficheiro Nenhum ficheiro selecionado

O ZIP deve conter os ficheiros multimédia (MP3/WAV/JPG/PNG) e um ficheiro `entidades.xml`.

[Enviar Lote \(ZIP\)](#)

Os Meus Uploads

Título	Tipo	Estilo	Distrito	Visibilidade	Data
Sai-te de cor	Áudio	Acústica	Braga	Público	Guardar 05/06/2026 18:44
Parte-me o Pescopo	Áudio	Rock	Lisboa	Privado	Guardar 05/06/2026 18:22
Parte-me o Pescopo	Áudio	Hip-Hop	Lisboa	Público	Guardar 05/06/2026 18:15
Parte-me o Pescopo	Áudio	Hip-Hop	Lisboa	Público	Guardar 05/06/2026 18:13

☐ Anunciar um Concerto?

[— Voltar à página inicial](#)

Figura 13 - Paine de Uploads Antigos do Artista

O processo de upload inclui as seguintes validações de segurança: verificação da extensão do ficheiro contra uma lista de extensões permitidas; geração de um nome de ficheiro único com `uniqid()` para evitar sobreposições e impedir que o nome original do ficheiro do utilizador seja exposto no servidor; criação automática da pasta `uploads/` caso não exista; e inserção do registo na tabela `conteudos` com o caminho relativo para o ficheiro.

6.7. Perfil do Artista (perfil.php)

Antes de poder efetuar o seu primeiro upload, qualquer utilizador com perfil `simpatizante` é obrigado a preencher o seu perfil de artista através da página `perfil.php`. Esta obrigatoriedade é imposta pelo próprio sistema: quando um Simpatizante tenta aceder ao `artista_dashboard.php` sem ter o distrito base definido (`id_distrito = NULL`), é automaticamente redirecionado para `perfil.php`.

Nesta página, o artista seleciona o seu **Distrito Base** a partir de uma lista gerada dinamicamente a partir da tabela `categorias`. Esta escolha é gravada diretamente no campo `id_distrito` da tabela `utilizadores`, ficando associada permanentemente ao perfil do artista. O distrito pode ser alterado em qualquer momento através da mesma página.

Esta abordagem elimina a necessidade de o artista selecionar o distrito de cada *upload* individual, tornando o processo mais fluído e garantindo que toda a música publicada fica automaticamente categorizada na região correta.

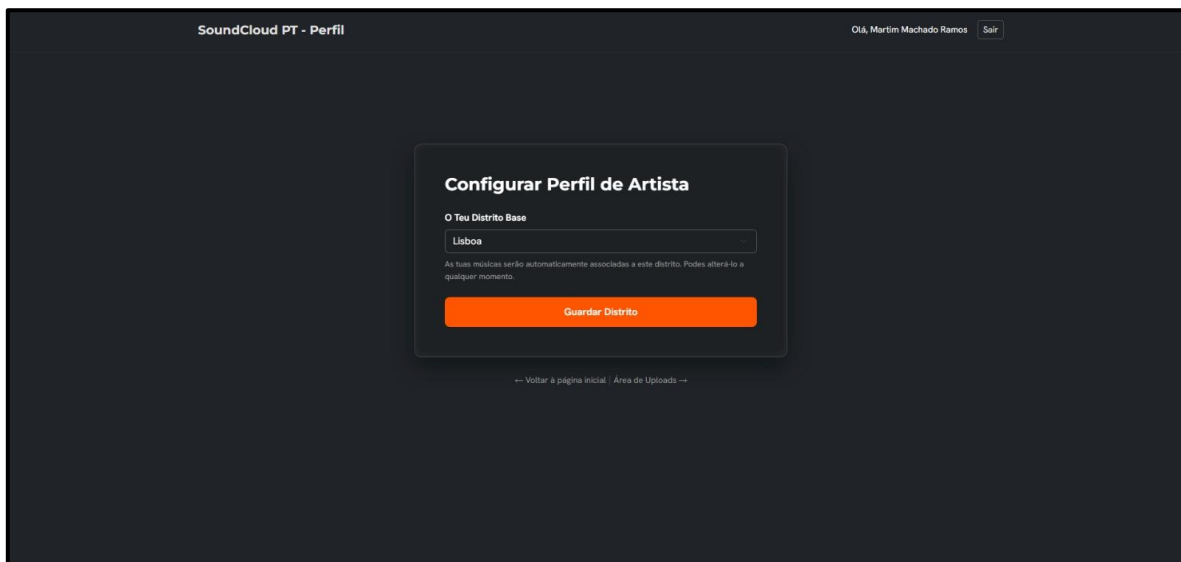


Figura 14 - Painel de Perfil - Escolha do Distrito que Pode Ser Alterado

6.8. Montra de Músicas de Filtros (index.php atualizado)

A página inicial foi completamente redesenhada para deixar de ser uma simples `landing page` e passar a funcionar como uma montra de conteúdos multimédia. A página apresenta agora os conteúdos publicados na plataforma sob a forma de *cards*, cada um com a capa do álbum, do título da faixa, o nome do artista, o distrito e o estilo

musicais associados, e um *player* de áudio HTML5, integrado que permite ouvir a música diretamente no *browser* sem necessidade de *download*.

6.8.1. Controlo de Visibilidade

O mecanismo de visibilidade está totalmente ativo na página inicial. Utilizadores não autenticados (Convidados) apenas visualizam conteúdos marcados como `publico`. Utilizadores com sessão ativa têm acesso adicional aos conteúdos marcados como `privado`. Esta filtragem é realizada inteiramente do lado do servidor, na *query* MySQL, e não pode ser contornada pelo *browser*.

6.8.2. Motor de Pesquisa e Filtros

No topo da página existe uma barra de filtros que permite a qualquer utilizador, autenticado ou não, refinar os conteúdos apresentados. Os filtros disponíveis são:

- **Pesquisa textual** por título ou nome de artista;
- **Filtro por Distrito** – lista gerada dinamicamente a partir da tabela `categorias`;
- **Filtro por Estilo Musical** – lista gerada a partir da tabela `estilos_musicais`;
- **Filtro por Tipo de Ficheiro** – áudio ou imagem.

Todos os filtros funcionam exclusivamente através de parâmetros `$_GET` no URL do browser, sem necessidade de recarregar a página com dados de sessão. Isto significa que qualquer resultado filtrado tem um URL partilhável diretamente.

6.9. Upload em Lote – ZIP + XML (`artista_dashboard.php`)

O mecanismo de *upload* em lote, identificado nos requisitos do projeto como a principal diferença face a sistemas genéricos como o *WordPress*, foi implementado integralmente. Esta funcionalidade permite a um artista publicar um EP ou álbum completo numa única operação, em vez de fazer o *upload* faixa a faixa.

Figura 15 - Painele de Upload do Artista

6.9.1. Estrutura do Pacote ZIP

O artista prepara um ficheiro .zip com a seguinte estrutura:

- Ficheiros áudio (MP3 ou WAV) correspondentes a cada faixa;
- Ficheiros de imagem opcionais (JPG ou PNG) para as capas individuais de cada faixa;
- Um ficheiro metadados.xml obrigatório na raíz do ZIP, com a metainformação de cada faixa.

6.9.2. Estrutura do Ficheiro XML

O ficheiro `metadados.xml` segue uma estrutura hierárquica onde cada elemento `<faixa>` referencia os ficheiros pelo nome e define os meta dados associados, incluindo título, descrição e estilo musical.

6.9.3. Processamento no Servidor

O motor de processamento em PHP executa as seguintes etapas sequencialmente:

1. Validação do ficheiro .zip recebido e extração para uma pasta temporária no servidor;
2. Leitura e *parsing* do `metadados.xml` com `SimpleXML`;
3. Para cada faixa descrita no XML, localização do ficheiro de áudio correspondente na pasta temporária;
4. Resolução do estilo musical – se o estilo indicado no XML não existir na tabela `estilos_musicais`, é criado automaticamente;

5. Geração de nomes únicos com `uniqid()` para os ficheiros de áudio e de capa;
6. Movimentação dos ficheiros para a pasta definitiva `uploads/`;
7. Inserção de um registo na tabela `conteudos` por cada faixa processada;
8. Limpeza da pasta temporária após a conclusão do processo.

Em caso de erro num ficheiro específico, o sistema regista o erro, continua o processamento das restantes faixas e apresenta um relatório final ao artista com o resultado de cada faixa (publicada com sucesso ou erro).

6.10. Mapa de Concertos (`index.php` + `artista_dashboard.php`)

A plataforma integra um serviço de mapa interativo para visualização e publicação de concertos, utilizando tecnologias de mapeamento *open-source* que não requerem chave de API paga.

6.10.1. Tecnologias Utilizadas

- **Leaflet.js (carregado via CDN):** biblioteca JavaScript para renderização de mapas interativos no browser;
- **OpenStreetMap Tiles:** imagens cartográficas de mapa, gratuitas e sem restrições de utilização;
- **Nominatim API:** serviço de geocodificação open-source que converte um texto de localização (ex: "Hard Rock Café, Lisboa") em coordenadas GPS (latitude e longitude).

6.10.2. Anunciar um Concerto

No *dashboard* do artista existe uma secção dedicada à gestão de concertos. Ao assinalar a opção "Anunciar um Concerto", é revelado um formulário com dois campos: o nome do local em texto livre e a data e hora do evento. Ao submeter, o JavaScript do lado do cliente chama a API Nominatim para resolver as coordenadas geográficas do local antes de enviar os dados ao servidor. O PHP valida os dados recebidos e insere um registo na tabela `concertos`. O artista pode também consultar e remover os seus concertos anunciados através de uma tabela no mesmo painel.

✓

Anunciar um Concerto?

Local do Concerto

Ex: Hard Rock Cafe, Lisboa

Data e Hora

dd/mm/aaaa --:--

Escreve o nome do local e a cidade. O sistema irá localizá-lo automaticamente no mapa.

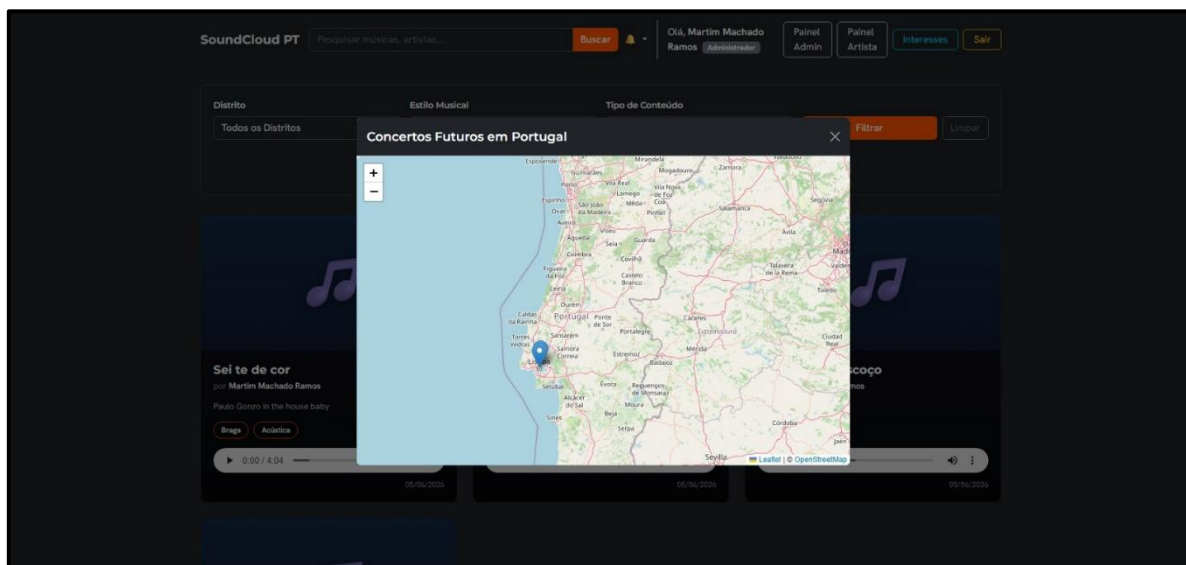
Publicar Concerto

Os Meus Concertos Anunciados

Local	Data	Ação
Instituto Superior de Engenharia de Lisboa	10/06/2026 21:13	Remover
Coliseu dos Recreios, Lisboa	13/06/2026 23:15	Remover

←

Voltar à página inicial



7. Segurança

A segurança foi uma preocupação transversal a todo o desenvolvimento da plataforma. As medidas implementadas seguem boas práticas consolidadas de desenvolvimento web seguro, descritas de seguida.

7.1. Encriptação de Palavras Chave

As palavras-passe dos utilizadores nunca são guardadas em texto simples na base de dados. No momento do registo, a palavra-passe é encriptada com o algoritmo `bcrypt` através da função nativa do PHP `password_hash()`, que gera automaticamente um *salt* aleatório para cada hash. No momento do login, a verificação é feita com `password_verify()`, que compara a palavra-passe introduzida com o *hash* guardado sem nunca desencriptar o original. Desta forma, mesmo que a base de dados seja comprometida, as palavras-passe dos utilizadores permanecem protegidas.

7.2. Princípio do Menor Privilégio

Todas as novas contas criadas na plataforma recebem automaticamente o perfil mínimo (*utilizador*) independentemente do que o utilizador introduza no formulário de registo. A promoção para perfis superiores (*simpatizante* ou *administrador*) só pode ser realizada por um administrador através do painel de controlo. Esta abordagem garante que nenhum utilizador consegue escalar privilégios por si próprio, minimizando a superfície de ataque do sistema.

7.3. Autenticação de Dois Fatores por E-mail

Após o registo, a conta é criada no estado inativo e o login é bloqueado até que o utilizador confirme o seu endereço de e-mail. O sistema gera um *token* único de 64 caracteres com `bin2hex(random_bytes(32))`, uma fonte de aleatoriedade criptograficamente segura, e envia-o por e-mail como parte de um link de ativação. Após a ativação, o *token* é eliminado da base de dados, tornando o link de ativação inválido para utilizações futuras. Esta medida garante que todos os endereços de e-mail registados são válidos e pertencentes ao utilizador, prevenindo registos com e-mails falsos.

7.4. Proteção Contra Spam — Google reCAPTCHA v2

O formulário de registo integra a API Google reCAPTCHA v2, que apresenta ao utilizador o desafio "Não sou um robô". A validação do token gerado pelo reCAPTCHA é processada do lado do servidor, no *backend* em PHP, através de um pedido à API da

Google antes de qualquer operação na base de dados ser executada. Isto garante que a validação não pode ser contornada por manipulação do lado do cliente.

7.5. Isolamento de Credenciais (`credenciais.php`)

Todas as chaves privadas do sistema, credenciais de acesso à base de dados, chave secreta do reCAPTCHA e credenciais SMTP do servidor de e-mail, estão isoladas num único ficheiro `credenciais.php`, separado da lógica da aplicação. Este ficheiro é mantido fora de qualquer partilha pública do código-fonte, garantindo que dados sensíveis não ficam expostos.

7.6. Controlo de Acesso por Perfil

Todas as páginas restritas verificam no início do ficheiro se existe uma sessão ativa com o perfil adequado. Por exemplo, `admin_dashboard.php` redireciona imediatamente para `index.php` qualquer utilizador que não tenha o perfil `administrador`, mesmo que tente aceder ao URL diretamente. O mesmo mecanismo é aplicado em `artista_dashboard.php` para os perfis `simpatizante` e `administrador`. Esta verificação é feita do lado do servidor, não podendo ser contornada por manipulação do *browser*.

7.7. Validação e Segurança no Upload de Ficheiros

O motor de upload em `artista_dashboard.php` implementa as seguintes proteções: verificação da extensão do ficheiro contra uma lista branca de extensões permitidas (mp3, wav, jpg, png), rejeitando qualquer outro tipo de ficheiro; geração de um nome de ficheiro único com `uniqid()` no momento do upload, eliminando a possibilidade de sobreposição de ficheiros e impedindo que o nome original do ficheiro do utilizador seja exposto no servidor.

Tabela 3 - Proteções Implementadas e o Seu Objetivo

Medida de Segurança	Vetor de Ataque Mitigado
<code>bcrypt</code> com <code>password_hash()</code>	Roubo de base de dados / <i>Password cracking</i>
Princípio do menor privilégio	Escalada não autorizada de privilégios
2FA por e-mail com <i>token</i> seguro	Registos com e-mails falsos
Google reCAPTCHA v2 (<i>backend</i>)	Registo automatizado por <i>bots</i>
Isolamento de credenciais	Exposição de chaves privadas no repositório
Controlo de acesso por perfil (<i>server-side</i>)	Acesso direto a URLs restritos
Lista branca de extensões no <i>upload</i>	<i>Upload</i> de ficheiros maliciosos

8. Discussão de Alternativas

Durante o desenvolvimento da plataforma, várias decisões técnicas foram tomadas entre múltiplas opções disponíveis. Esta secção apresenta as principais alternativas consideradas e justifica as opções finais adotadas.

8.1. Linguagem de Servidor: PHP vs Alternativas

A linguagem de servidor adotada foi o PHP, por ser o requisito definido no enunciado e por ser a linguagem nativa do ambiente de desenvolvimento com Apache e MySQL. No entanto, existem alternativas modernas igualmente válidas para o mesmo tipo de sistema:

- **Node.js com Express:** permite desenvolver o servidor e o cliente na mesma linguagem (JavaScript), com desempenho elevado em operações de I/O. No entanto, a sua configuração é mais complexa e a integração com Apache não é direta.
- **Python com Django ou Flask:** oferece uma sintaxe mais limpa e frameworks com funcionalidades avançadas de segurança integradas. Contudo, exige um servidor WSGI dedicado e tem menor penetração em ambientes de alojamento partilhado comparativamente ao PHP.

O PHP foi mantido por cumprir os requisitos do projeto, pela sua integração nativa com Apache e MySQL, e pela vasta disponibilidade de documentação e suporte.

8.2. Base de Dados: MySQL vs Alternativas

O MySQL foi a escolha adotada, alinhada com os requisitos do projeto. As principais alternativas seriam:

- **PostgreSQL:** sistema de gestão de base de dados relacional mais robusto e com suporte a tipos de dados mais avançados. Seria uma escolha superior em termos técnicos para um sistema em produção, mas a sua configuração no ambiente Docker utilizado seria mais complexa e o ganho não justificaria a mudança para o âmbito deste projeto.
- **SQLite:** base de dados baseada em ficheiro, sem necessidade de servidor dedicado. Adequada para protótipos de pequena escala, mas sem suporte a acessos concorrentes, tornando-a inadequada para uma plataforma multiutilizador.

O MySQL foi mantido por ser a solução mais equilibrada entre simplicidade de configuração, desempenho e suporte nativo no ambiente LAMP utilizado.

8.3. Armazenamento de Ficheiros Multimédia: Sistema de Ficheiros vs Base de Dados

Uma alternativa ao armazenamento dos ficheiros multimédia na pasta `uploads/` seria guardar os ficheiros diretamente na base de dados, codificados em Base64 ou como campos `BLOB`. Esta abordagem foi descartada pelos seguintes motivos:

- Guardar ficheiros binários na base de dados aumenta significativamente o tamanho da BD e degrada o desempenho das consultas;
- A abordagem de guardar o caminho do ficheiro na BD e o ficheiro no sistema de ficheiros é a prática padrão da indústria, adotada por sistemas como o WordPress;
- A gestão, cópia de segurança e distribuição de ficheiros estáticos é mais eficiente diretamente no sistema de ficheiros.

8.4. Envio de E-mail: PHPMailer vs mail() Nativa

O PHP disponibiliza a função nativa `mail()` para o envio de e-mails. No entanto, esta função foi preterida em favor da biblioteca **PHPMailer** pelas seguintes razões:

- A função `mail()` não suporta autenticação SMTP, tornando o envio de e-mails dependente da configuração do servidor, o que não é garantido em todos os ambientes;
- O PHPMailer suporta SMTP com autenticação e SSL/TLS, tornando o envio de e-mails compatível com qualquer servidor de e-mail moderno, incluindo o Gmail;
- O PHPMailer oferece melhor controlo sobre os cabeçalhos do e-mail, reduzindo a probabilidade de as mensagens serem classificadas como spam.

8.5. Framework de Interface: Bootstrap vs CSS Personalizado

Para o desenvolvimento da interface, foi adotado o **Bootstrap** como *framework* de estilos. A alternativa seria desenvolver o CSS completamente de raiz. As vantagens do Bootstrap neste contexto foram:

- Responsividade automática para dispositivos móveis e *desktop* sem esforço adicional;
- Componentes prontos a usar (formulários, botões, tabelas, grelhas) que aceleram o desenvolvimento;
- Consistência visual garantida em todos os browsers modernos.

A desvantagem do Bootstrap é a menor personalização visual em comparação com CSS desenvolvido de raiz. Para um projeto académico com foco na lógica de *backend*, esta troca é amplamente justificada.

8.6. Upload Unitário vs Upload em Lote (ZIP + XML)

O sistema implementado suporta tanto o *upload* unitário como o *upload* em lote. No mecanismo de upload em lote, o artista empacota múltiplos ficheiros de áudio e as respetivas capas num único ficheiro `.zip`, acompanhado de um ficheiro `metadados.xml` com o título, descrição e estilo musical de cada faixa. O motor *backend* em PHP extrai o pacote, faz o *parsing* do XML com `SimpleXML`, cruza automaticamente os dados com os ficheiros correspondentes, resolve estilos musicais inexistentes criando-os dinamicamente, e publica todas as faixas numa única operação. Esta funcionalidade cumpre integralmente o requisito de envio em lote definido no enunciado do projeto.

9. Conclusões e Trabalho Futuro

9.1. Conclusões

O presente projeto permitiu desenvolver, de raiz, um Sistema de Gestão de Conteúdos multimédia funcional, assente numa arquitetura cliente/servidor com PHP, Apache e MySQL. A plataforma cumpre os principais requisitos definidos no enunciado: suporte a múltiplos perfis de utilizador com permissões diferenciadas, upload e gestão de conteúdos multimédia com controlo de visibilidade, organização hierárquica de conteúdos por categorias, e um conjunto robusto de medidas de segurança.

O tema escolhido, um "SoundCloud Português" dedicado a artistas independentes, revelou-se adequado para ilustrar todos os conceitos trabalhados na unidade curricular, nomeadamente a gestão de sessões, o controlo de acesso por perfil, o upload seguro de ficheiros e a integração com serviços externos como a API do Google reCAPTCHA e o serviço de e-mail via SMTP.

A comparação com o WordPress, apresentada no **CAPÍTULO 2 - ENQUADRAMENTO E TRABALHOS RELACIONADOS**, demonstrou que a arquitetura adotada segue os mesmos princípios dos sistemas de gestão de conteúdos mais utilizados no mundo, validando as opções técnicas tomadas. A principal diferença reside no facto de o sistema desenvolvido ter sido construído de raiz para um caso de uso específico, o que permite um controlo total sobre a lógica de negócio, a estrutura de dados e as políticas de segurança aplicadas.

De forma geral, os objetivos definidos na introdução foram cumpridos, incluindo o mecanismo de *upload* em lote (ZIP + XML) .a integração de um serviço externo de mapeamento para concertos, e o sistema de notificações e subscrições por distrito e estilo musical.

9.2. Trabalho Futuro

O sistema desenvolvido constitui uma base sólida sobre a qual podem ser construídas funcionalidades adicionais. As principais melhorias identificadas para fases seguintes são:

Alertas RSS: criação de *feeds* RSS por categoria e por artista, permitindo que os utilizadores sigam as novidades da plataforma através de qualquer leitor de RSS, sem necessidade de conta registada.

Partilha em Redes Sociais: integração de botões de partilha para Facebook e Twitter em cada página de conteúdo, permitindo que os artistas promovam as suas obras diretamente a partir da plataforma.

.

10. Bibliografia

Linguagem e Servidor

PHP Group. (2024). *PHP Manual*. <https://www.php.net/manual/en/>

Apache Software Foundation. (2024). *Apache HTTP Server Documentation*. <https://httpd.apache.org/docs/>

Base de Dados

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>

Framework de Interface

Bootstrap Team. (2024). *Bootstrap 5 Documentation*. <https://getbootstrap.com/docs/5.0/>

Biblioteca de E-mail

PHPMailer Contributors. (2024). *PHPMailer — A full-featured email creation and transfer class for PHP*. GitHub. <https://github.com/PHPMailer/PHPMailer>

Segurança

Google Developers. (2024). *reCAPTCHA v2 Developer Guide*. <https://developers.google.com/recaptcha/docs/display>

OWASP Foundation. (2024). *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>

Sistemas de Referência

WordPress Foundation. (2024). *WordPress Developer Documentation*. <https://developer.wordpress.org/>

SoundCloud GmbH. (2024). *SoundCloud*. <https://soundcloud.com/>

Ambiente de Desenvolvimento

Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com/>